

C++ Programming Textbook Notes

1. Overview of Data Types in C++

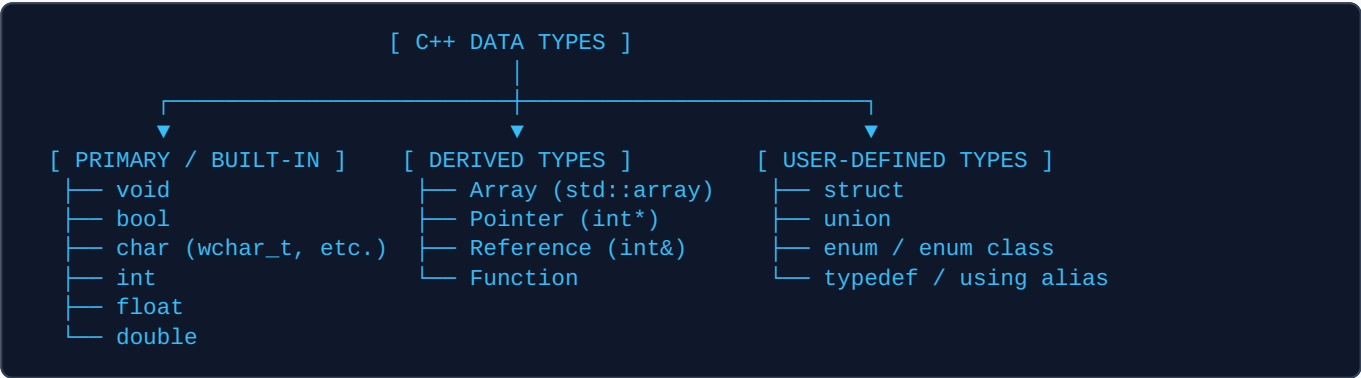
Conceptual Explanation

A **Data Type** is an explicit instruction to the compiler that specifies two critical things:

- Data Nature:** What kind of value is stored (e.g., text, whole numbers, decimals, truth values).
- Memory Allocation:** Exactly how much memory space (in bytes) to reserve in RAM and how bits should be interpreted.

In C++, data types ensure strict compile-time type safety, preventing invalid memory reads and mismatched operations.

Classification Hierarchy of Data Types



2. Primary Data Types in C++

Conceptual Explanation

Primary (built-in) data types form the foundational building blocks of C++. Unlike C, C++ introduces native boolean support (`bool`) and wide character extensions natively into its primary family.

Core Primary Types Table

Data Type	Used For	Typical Memory Size	C++ Example
<code>bool</code>	Boolean truth values (<code>true</code> / <code>false</code>)	1 Byte	<code>bool isPassed = true;</code>
<code>char</code>	Single characters or ASCII codes	1 Byte (8 bits)	<code>char grade = 'A';</code>
<code>int</code>	Whole numbers (no decimals)	4 Bytes (32 bits)	<code>int score = 42;</code>
<code>float</code>	Single-precision floating-point numbers	4 Bytes (32 bits)	<code>float weight = 3.14f;</code>
<code>double</code>	Double-precision floating-point numbers	8 Bytes (64 bits)	<code>double pi = 3.1415926535;</code>
<code>void</code>	Absence of type / empty return	0 Bytes (N/A)	<code>void printMsg();</code>

Text-Based Memory Representation Diagram

RAM Bit Layouts:

```
char (1 Byte / 8 bits):  
[ 0 ][ 1 ][ 0 ][ 0 ][ 0 ][ 0 ][ 0 ][ 1 ] → Represents ASCII 'A' (65)  
  
int (4 Bytes / 32 bits):  
[00000000][00000000][00000000][00101010] → Represents Decimal 42  
  
double (8 Bytes / 64 bits):  
[ Sign (1b) ][ Exponent (11b) ][ Mantissa / Fraction (52b) ]
```

Primary Data Types Code Example

```
#include <iostream>  
  
int main() {  
    bool active = true;  
    char initial = 'G';  
    int count = 100;  
    float temp = 36.6f;  
    double distance = 149600000.75;  
  
    std::cout << "Bool: " << std::boolalpha << active << "  
";  
    std::cout << "Char: " << initial << "  
";  
    std::cout << "Int: " << count << " (" << sizeof(count) << " bytes)  
";  
    std::cout << "Float: " << temp << " (" << sizeof(temp) << " bytes)  
";  
    std::cout << "Double: " << distance << " (" << sizeof(distance) << " bytes)  
";  
  
    return 0;  
}
```

⚠ Floating-Point Precision Gotcha: Floating-point types (float and double) cannot represent all decimal fractions exactly due to IEEE 754 binary representation. Never compare floating-point values using ==.

3. Derived and User-Defined Data Types

Conceptual Explanation

- **Derived Data Types:** Created by referencing or grouping basic types (e.g., Arrays, Pointers, Functions, and C++ References).
- **User-Defined Data Types:** Custom types designed by programmers to group complex real-world data structures (e.g., struct, union, enum/enum class, and using aliases).

Derived & User-Defined Summary Table

Category	Type	Purpose & Concept
Derived	Array	Sequential collection of homogeneous items stored in contiguous memory.
Derived	Pointer	Variable holding raw physical RAM address of another variable.
Derived	Reference (&)	C++ specific alias or secondary name for an existing variable location.
User-Defined	struct / class	Groups heterogeneous data members under one logical name.
User-Defined	union	Shares a single memory location among all member variables.
User-Defined	enum class	Scoped, strongly-typed enumeration of named integer constants.
User-Defined	using / typedef	Creates a custom shortcut or type alias.

4. Type Qualifiers in C++

Conceptual Explanation

A **Type Qualifier** modifies the size, range, sign interpretation, or mutability of a base data type.

Think of a base type as a standard *vehicle* (e.g., `int` = Car). Applying qualifiers is like specifying extra attributes: *Compact Car* (`short int`), *Heavy-duty Truck* (`long int`), or *Armored Sealed Vehicle* (`const int`).

Core Type Qualifiers Matrix

Qualifier	Function & Memory Impact	Compatible Base Types
short	Reduces memory size (typically 2 Bytes)	int
long	Expands memory size/range (4 or 8 Bytes)	int, double
signed	Allows positive and negative values (Default for int/char)	char, int
unsigned	Restricts to non-negative values, doubling positive range	char, int
const	Locks value to prevent modifications after initialization	All data types
volatile	Prevents compiler optimizations on variables changed externally	All data types

Unsigned Overflow Behavior (Wrap-around Logic)

Unsigned Integer Wrap-around (8-bit example, 0 to 255):

0 → 1 → ... → 254 → 255

↑

└────────── (+1) ─────────┘ (Wrap-around to 0 upon overflow!)

Qualifiers Code Example

```
#include <iostream>

int main() {
    short int age = 20;           // Uses 2 bytes
    long long int population = 14000000000LL; // Uses 8 bytes
    unsigned int totalStudents = 500; // Only positive values (0 to ~4.2 Billion)
    const double pi = 3.14159;    // Read-only constant

    std::cout << "Short Int Size: " << sizeof(age) << " bytes\n";
    std::cout << "Long Long Size: " << sizeof(population) << " bytes\n";
    std::cout << "Unsigned Size: " << sizeof(totalStudents) << " bytes\n";

    return 0;
}
```

⚠️ Unsigned Underflow Bug: Decrementing an unsigned variable past 0 wraps it around to its maximum possible value (e.g., unsigned int x = 0; x--; results in 4,294,967,295). Avoid using unsigned types in standard decrementing loops!

5. Architecture Memory Size Reference Table

Data Type / Qualification	Typical Modern Size	Valid Range / Description
char	1 Byte (8 bits)	-128 to 127 or 0 to 255
unsigned char	1 Byte (8 bits)	0 to 255 (Raw byte manipulation)
short int	2 Bytes (16 bits)	-32,768 to 32,767
unsigned short int	2 Bytes (16 bits)	0 to 65,535
int	4 Bytes (32 bits)	-2,147,483,648 to 2,147,483,647
unsigned int	4 Bytes (32 bits)	0 to 4,294,967,295
long long int	8 Bytes (64 bits)	-9.22 × 10 ¹⁸ to 9.22 × 10 ¹⁸
float	4 Bytes (32 bits)	~7 decimal digits precision
double	8 Bytes (64 bits)	~15-17 decimal digits precision
Pointer (type*)	4 Bytes (32-bit OS) / 8 Bytes (64-bit OS)	Scales automatically based on CPU architecture